

SET-I

Q1. List any ten Java command tools with a short description of each.

Ans.

Ten commonly used Java command tools available in the Java Development Kit (JDK) along with a short description of each are listed below:

- i. `javac`: This command is used to compile Java source files (Java programs) into binary class files known as bytecode.
- ii. `java`: This command runs Java bytecode to launch a Java application.
- iii. `javadoc`: This tool generates API documentation from Java source code.
- iv. `jar`: This tool is used to aggregate and compress multiple files into a single JAR (Java ARchive) file.
- v. `javap`: This command disassembles Java files and prints out a representation of the Java bytecode.
- vi. `jdb`: This is a debugger tool that helps to find and fix problems in a Java class.
- vii. `appletviewer`: This tool is used to run and view applets without requiring a Web browser.
- viii. `javah`: This creates C-language header and stub files from a Java class, which allows Java and C code to interact.
- ix. `jarsigner`: This tool is used to digitally sign a jar file and to verify the integrity and signatures of signed jar files.
- x. `keytool`: This tool is used to create, manage, and store keys and security certificates.

Q2. Write Java program to convert the Rupees to Dollars.

Ans.

Here is a Java program to convert Rupees to Dollars:

```
import java.util.Scanner;

public class RupeeToDollar {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter amount in Rupees: ");
        double rupees = scanner.nextDouble();
        double conversionRate = 90.0;
        double dollars = rupees / conversionRate;
        System.out.println(rupees + " Rupees is equal to " + dollars + " Dollars.");
    }
}
```

Explanation of the Code:

- i. **Class and Main Method:** The program is defined within a class (named `RupeeToDollar`), and the execution begins with the `main()` method.
- ii. **Data Types:** The `double` data type is used for the variables `rupees`, `conversionRate`, and `dollars`. In Java, `double` is a primitive data type used for double-precision floating-point numbers, which is suitable for currency calculations that may require decimal values.
- iii. **Arithmetic Operators:** The program utilizes the division operator (`/`) to calculate the dollar amount by dividing the rupee input by the conversion rate. Arithmetic operators are used to perform mathematical operations on operands,.
- iv. **Input:** The `Scanner` class is used to capture user input from the system, a feature detailed in later units of the text.
- v. **Output:** The `System.out.println` method is used to display the final calculated string on the screen.

Q3. What are the different methods under `DataInputStream` and `DataOutputStream`?

Ans.

In the Java Input/Output (I/O) system, streams are used to handle the flow of data, and they are categorized into byte streams and character streams. Within the byte stream category, `DataInputStream` and `DataOutputStream` are specific classes known as filter streams. These classes are distinct because they allow for the reading and writing of Java primitive data types, rather than just handling raw bytes.

DataInputStream Methods The `DataInputStream` class provides the capability to read primitive data types from an input stream. It accomplishes this by implementing the methods defined in the `DataInput` interface. Key methods available in this class include:

- i. `boolean readBoolean():` This method reads a single byte from the input stream. It interprets the byte to determine a boolean value, returning `true` if the byte is nonzero and `false` if it is zero.
- ii. `byte readByte():` This method reads a byte from the stream and treats it as an 8-bit signed value.
- iii. `char readChar():` This method is used to read a Unicode character from the stream. This is essential because Java represents characters using Unicode (16 bits) rather than simple ASCII (8 bits).
- iv. `int readInt():` This method reads an integer value from the input stream.

DataOutputStream Methods The `DataOutputStream` class acts as a complement to the `DataInputStream` class. It provides methods to write Java primitive data types to an output stream. Additionally, this class maintains a record of the number of bytes that have been written to the output stream. The primary methods for this class include:

- i. `void writeChar(int v):` This method writes a character to the output stream. It accepts an integer `v` as a parameter, which represents the character to be written.
- ii. `void writeLong(long v):` This method is used to write a long integer to the output stream. It takes a long value `v` as its argument.

- iii. `void writeInt(int v)`: This method writes an integer to the output stream. It takes an int value v as the data to be written.

SET-II

Q4. What is the difference between manual and automated testing?

Ans.

The differences between manual and automated testing are:

Manual Testing

- i. Process: Manual testing is performed by developers manually. In the context of unit testing, this requires the programmer to test each section of the code by hand. If a change or addition is made, the programmer must compile the code and test every portion again to ensure the flow is not broken and the results are as expected.
- ii. Suitability: It works well for small applications where not many changes are required. It is also preferred for short codes and bugs that can be easily detected without writing specific code to run them.
- iii. Advantages:
 - a) Bugs can be easily found and removed.
 - b) Programmers can spend more time refining their code.
 - c) New test cases can be executed swiftly.
 - d) It supports ad-hoc testing.
 - e) It is initially cheaper than automated testing because it requires less investment at the beginning.
- iv. Disadvantages: It is time-consuming and requires more effort than necessary. For complex or lengthy applications, it is not feasible to manually run the whole application through tests every time a small change is made, making it tiresome for programmers.

Automated Testing

- i. Process: Automated testing involves writing a set of code or instructions to let the computer perform the work. It is performed by testing tools. This method tests the application or code block-by-block every time the programmer compiles the code or makes a change.
- ii. Suitability: It is introduced for bigger, complex, and lengthy applications where manual testing is not feasible. It is particularly useful for codes that require frequent edits or changes.
- iii. Advantages:
 - a) It reduces the effort and time required by the programmer.
 - b) Once the test code is written, it saves a significant amount of time and resources.
 - c) It ensures that the application is bug-free.

Q5. Explain the purpose and functionality of the JList component in Java Swing. How does it differ from other list-type components?

Ans.

Purpose and Functionality of JList component in Java Swing:

The primary purpose of the JList component is to display a selectable list of choices to the user. While these choices are most commonly strings or images, the component is designed with enough flexibility to display arbitrary objects. This capability makes JList a versatile tool for presenting data sets where the user needs to select one or multiple items.

Functionally, JList operates as part of the broader Swing architecture. Like all Swing components, it inherits from the JComponent class, which serves as the root of the Swing component hierarchy. This inheritance allows JList to utilize standard Swing features, such as tooltips and double-buffering.

A critical aspect of the JList functionality is its integration with Java's event-handling mechanism. The component relies on the javax.swing.event package to monitor user interactions. Specifically, when a user selects a particular item within the list, the JList class utilizes the classes provided in the event package to determine the selection and allow the application to respond accordingly.

Differences from Other List-Type Components

To understand the specific role of JList, it is helpful to distinguish it from other components and structures that manage lists of data:

- i. **Difference from JComboBox:** The Swing library includes another component called JComboBox, which also handles list selection but functions differently. A JComboBox is described as a combination of a text entry field and a drop-down list of choices. This allows a user to either type a selection or choose one from a collapsed list. In contrast, a JList typically displays multiple options simultaneously in an expanded format and does not inherently include a text entry field for user input. Therefore, JList is preferable when screen space allows for displaying multiple items at once, whereas JComboBox is better suited for compact layouts.
- ii. **Difference from the List Interface (Collections):** It is crucial to differentiate the JList visual component from the List interface found in the Java Collections Framework. The List interface (implemented by classes like ArrayList and LinkedList) represents a data structure—an ordered group of elements used for backend data manipulation, storage, and sorting.
- iii. **Conversely, JList is a UI mechanism used to display that data to the user.** While a JList may use a List or array to hold the data it presents, the JList itself is concerned with presentation and user selection events, whereas the List interface is concerned with data organization, duplicate handling, and algorithmic sequencing.

Q6. Differentiate between ArrayList and LinkedList. Provide suitable examples where each is preferred.

Ans.

ArrayList and LinkedList are both general-purpose implementations of the List interface, but they possess distinct internal structures and performance characteristics.

Differentiation:

- Internal Structure:

- ArrayList: This acts as a resizable array implementation. It allows for the storage of all elements, including NULL, and is similar to a Vector except that it is unsynchronized,.

- LinkedList: This is a linear data structure composed of a sequence of nodes. Each node contains data and a reference to the next node in the sequence.

- Access Performance:

- ArrayList: It provides constant-time positional access. This makes it highly efficient for random access and quick iteration.

- LinkedList: Positional access takes linear time, meaning the time to access an element increases with the size of the list.

- Modification Performance (Insertion/Deletion):

- ArrayList: Inserting or deleting elements from the inside of the list takes linear time. It may need to use System.arraycopy to move numerous elements at once when modifications occur.

- LinkedList: Operations such as adding elements to the beginning or removing elements from the inside take constant effort. It allows for efficient insertion and removal without the need to reallocate or reorganize the entire structure.

- Additional Features:

- ArrayList: Its only tuning option is the initial capacity, which dictates how many elements it can store before expansion is required.

- LinkedList: It implements the Queue interface and provides specific methods like addFirst, removeFirst, addLast, removeLast, getFirst, and getLast.

Suitable Examples for Preference

i. ArrayList Preference: ArrayList should be preferred when your application requires fast, random access to elements using an index. It is generally the default choice because it is "just straight quick" for positional references. For example, a list of static items that are read frequently but rarely modified would benefit from ArrayList.

ii. **LinkedList Preference:** LinkedList should be preferred if the application involves frequently adding members to the beginning of the list or iterating through the list to delete elements from the inside. For example, implementing a queue or a stack where elements are constantly added or removed from the ends would be more efficient with LinkedList due to its dynamic nature.